CHƯƠNG 3: CÀI ĐẶT THỰC NGHIỆM VÀ KẾT QUẢ

Trên cơ sở mô hình mật mã lai ở Chương 1 và đặc tả hệ thống tại Chương 2, chương này trình bày việc hiện thực các mô-đun bằng Node.js, TypeScript và NestJS. Hai đơn vị giữ cặp khóa RSA riêng, trao đổi phong bì JSON qua REST API và quản lý danh mục khóa công khai, tài liệu đã nhận bằng MongoDB. Mỗi kết quả kiểm thử, ảnh giao diện và số liệu hiệu năng được lấy từ bản chương trình đã chạy, làm cơ sở đánh giá mức độ đáp ứng yêu cầu thiết kế.

3.1. Môi trường và công nghệ triển khai

3.1.1. Lựa chọn ngôn ngữ lập trình và nền tảng phát triển

TypeScript được sử dụng để khai báo rõ kiểu dữ liệu của header, payload và phong bì, đồng thời bật chế độ strict để phát hiện lỗi kiểu trước khi chạy. NestJS tổ chức chương trình theo controller, service và module, thuận tiện cho việc tách xử lý HTTP khỏi logic mật mã. Cùng một mã nguồn được khởi động với UNIT_ID, DATA_DIR và PORT khác nhau để tạo đơn vị A và B.

Thành phần	Cấu hình thực nghiệm
Hệ điều hành / kiến trúc	darwin / arm64
Bộ xử lý	Apple M5, 10 CPU logic
Bộ nhớ RAM	24.0 GiB
Nền tảng	Node.js v22.22.3, OpenSSL 3.5.6
Framework / ngôn ngữ	NestJS 11.2.8, TypeScript 5.9 (strict)
Cơ sở dữ liệu	MongoDB 8.0.34, MongoDB Node.js Driver, GridFS
Tham số mật mã	RSA-2048; AES-256-CBC; OAEP/SHA-256; RSASSA-PKCS1-v1_5/SHA-256
Giới hạn	Tệp gốc tối đa 10 MiB; cửa sổ timestamp đối xứng ±300 giây theo mục 2.4.4

Hai tiến trình HTTP chạy trên cùng máy qua loopback để phục vụ học tập. Đây là mô phỏng hai đơn vị với kho khóa tách biệt, chưa phải thử nghiệm truyền dẫn giữa hai máy vật lý hoặc đánh giá khả năng chịu tải trong môi trường triển khai thật.

3.1.2. Các thư viện mật mã học chuyên dụng

Mô-đun node:crypto có sẵn trong Node.js thực hiện phép toán mật mã thông qua OpenSSL [1]. Các API generateKeyPair, createCipheriv, publicEncrypt, sign và verify được sử dụng thay cho việc tự lập trình phép toán RSA hoặc các vòng biến đổi AES. Thư viện mongodb kết nối cơ sở dữ liệu và GridFS [6]. Multer tiếp nhận tệp multipart; class-validator kiểm tra DTO; @nestjs/swagger sinh giao diện thao tác từ controller [2, 3]. Bộ kiểm thử sử dụng node:test, node:assert và fetch để kiểm tra cả service lẫn giao tiếp HTTP thực tế.

Nhiệm vụ	**API được sử dụng**
Sinh khóa và số ngẫu nhiên	generateKeyPair(), randomBytes(), randomUUID()
AES-CBC	createCipheriv(), createDecipheriv(), update(), final()
Bọc / mở khóa phiên	publicEncrypt(), privateDecrypt(), RSA_PKCS1_OAEP_PADDING
Băm / ký / xác minh	createHash('sha256'), sign(), verify(), RSA_PKCS1_PADDING

Bảng 3.2 đối chiếu các yêu cầu thiết kế với thành phần hiện thực. Mỗi thao tác nhạy cảm được xử lý trong service, còn controller tiếp nhận HTTP và chuyển kết quả ra giao diện. Mã nguồn có chú thích // tiếng Việt trước từng hàm và tại các bước mật mã chính.

Yêu cầu Chương 2	**Thành phần hiện thực**	**Bằng chứng kiểm tra**
2.1.2: sinh, xuất/nhập PEM	CryptoService, KeysService, /api/keys/*	RSA 2048; nhập PKCS#1; lưu PKCS#8 mã hóa
2.2.2: danh mục khóa MongoDB	StorageService; collection peers	Truy vấn khóa và ghim vân tay
2.3: header/payload	envelope.ts; EnvelopesService.seal()	Test đúng 5 trường header, 3 trường payload
2.4: tiếp nhận HTTP JSON	POST /api/envelopes/receive	Gửi A → B, tải lại tệp từ GridFS
2.4.4: timestamp và UUID	Cửa sổ ±300 giây; unique ownerId/request_id	Hết cửa sổ, phát lại, sửa UUID, nhận đồng thời
2.1.3: tệp 10 MB dưới 1,5 giây	scripts/benchmark.ts	Đo thực tế có cả đọc/ghi MongoDB

3.2. Hiện thực hóa các mô-đun chức năng cốt lõi

Mã nguồn được chia thành src/crypto, src/keys, src/envelopes và src/storage. CryptoService cung cấp thao tác mật mã; KeysService quản lý PEM và danh mục đối tác; EnvelopesService điều phối việc đóng/mở gói; ReplayService chuyển việc ghi nhận UUID sang StorageService khi chạy với MongoDB. Controller tiếp nhận HTTP và chuyển yêu cầu đến service. Cấu hình chung đặt trong src/config.ts; bootstrap.ts tạo ứng dụng, giới hạn dung lượng và mở Swagger.

3.2.1. Mô-đun sinh và lưu trữ cặp khóa RSA

CryptoService.generateRsaPair() sinh khóa RSA với số mũ công khai 65537. API cho phép 2048, 3072 hoặc 4096 bit; các phép đo trong chương này sử dụng 2048 bit. Khóa công khai được xuất dưới dạng SPKI PEM. Khóa bí mật được xuất PKCS#8 PEM đã mã hóa bằng mật khẩu, lưu tại data/<đơn-vị>/identity/private.pem; quyền tệp trên POSIX là 0600. API sinh khóa chỉ trả khóa công khai và vân tay.

KeysService.generate() tạo thư mục nhận diện nguyên tử và không ghi đè cặp khóa hiện có. POST /api/keys/import cho phép nhập khóa PEM PKCS#1 hoặc PKCS#8; khóa được kiểm tra và chuẩn hóa thành PKCS#8 mã hóa trước khi lưu. GET /api/keys/public xuất khóa công khai. Vân tay là SHA-256 của SPKI DER. KeysService.trust() đối chiếu vân tay rồi lưu khóa vào collection peers trong MongoDB, với chỉ mục duy nhất theo ownerId và unitId. Hai đơn vị phải đối chiếu vân tay qua một kênh độc lập trước khi ghim khóa. Đây là danh mục khóa PEM, chưa phải hạ tầng chứng thực X.509 hoàn chỉnh.

3.2.2. Mô-đun mã hóa AES-256 và bọc khóa phiên RSA-OAEP

Tệp gốc được chuyển thành Base64 và đưa vào JSON bản rõ cùng trích yếu, tên tệp, loại nội dung, kích thước và SHA-256. Để bảo vệ thông tin điều khiển, JSON còn chứa protocol và control: bản sao năm trường header cùng IV. Khối bản rõ hoàn chỉnh được mã hóa AES-256-CBC với khóa phiên 32 byte và IV 16 byte do randomBytes() sinh mới cho từng gói. IV CBC cần khó dự đoán trước [4]; autoPadding mặc định của Node.js xử lý đệm PKCS#7. Bước bọc khóa được tính trước khi tạo JSON bản rõ để header.key cũng có bản sao trong control, không làm thay đổi thuật toán bọc khóa hay cấu trúc gói tin của Chương 2.

```
const cipher = createCipheriv('aes-256-cbc', key, iv);
const ciphertext = Buffer.concat([cipher.update(plaintext), cipher.final()]);
const encryptedKey = publicEncrypt({
  key: recipientPublicKey,
  padding: constants.RSA_PKCS1_OAEP_PADDING,
```

```
  oaepHash: 'sha256',
}, key);
```

RSA chỉ bọc khóa phiên AES thay vì mã hóa trực tiếp tài liệu. SHA-256 được chỉ định rõ cho OAEP; MGF1 sử dụng cùng hàm băm theo API Node.js. Bên nhận phải sử dụng cấu hình tương ứng khi gọi privateDecrypt(). Đặc tả RSA-OAEP được tham chiếu từ RFC 8017 [5].

3.2.3. Mô-đun băm SHA-256 và tạo chữ ký số

Theo đúng mục 2.3.3 và bước 5 của mục 2.4.2, chữ ký được tạo trên chuỗi Base64 encrypted_data. CryptoService.sign() gọi sign với SHA-256 và RSA_PKCS1_PADDING, tức RSASSA-PKCS1-v1_5. Thư viện tự tính SHA-256 nên chương trình truyền trực tiếp byte ASCII của encrypted_data, không băm trước rồi băm lại. Chữ ký được chuyển thành Base64 và đặt ở payload.signature. Khi xác minh, bên nhận tái tạo đúng các byte đầu vào này và dùng khóa công khai của sender_id lấy từ danh mục MongoDB.

```
const signature = sign('sha256', signingBytes(envelope), {
  key: senderPrivateKey,
  padding: constants.RSA_PKCS1_PADDING,
});
```

Hệ thống giữ mô hình Encrypt-then-Sign: có bản mã trước rồi mới tạo chữ ký. Cần phân biệt phạm vi xác thực: verify() xác minh trực tiếp encrypted_data, chưa xác nhận toàn bộ header và IV. Sau giải mã, open() kiểm tra protocol và đối chiếu từng trường header/IV với bản sao control trong bản rõ được mã hóa. Nếu có khác biệt, gói bị từ chối trước khi lưu tài liệu hoặc đánh dấu đã nhận. Phần đối chiếu bổ sung này giữ nguyên schema ngoài của Chương 2 và ngăn việc đổi request_id, timestamp để tái sử dụng bản mã hợp lệ. CBC tự nó không cung cấp tính xác thực.

3.2.4. Mô-đun đóng gói và giải nén JSON

EnvelopesService.seal() xuất phong bì gồm đúng hai khối header và payload như mục 2.3. Header có request_id, sender_id, receiver_id, timestamp, key. Payload có encrypted_data, iv, signature. Không thêm các trường expiresAt, senderKeyId hoặc tên thuật toán vào header. Bộ thuật toán được cố định trong mã nguồn. parseEnvelope() kiểm tra giới hạn dung lượng, UTF-8, tập trường chính xác, UUIDv4, thời gian UTC, Base64 chuẩn, IV 16 byte và độ dài bản mã là bội số của khối AES.

Bên nhận kiểm tra timestamp theo cửa sổ ±300 giây và receiver_id trước khi truy vấn khóa công khai theo sender_id. Sau đó verify() kiểm tra chữ ký trên

encrypted_data. Thay đổi thứ tự trường JSON hoặc khoảng trắng bên ngoài chuỗi Base64 không ảnh hưởng chữ ký. Các trường điều khiển được xác nhận lần cuối qua control sau khi giải mã; không coi một kết quả verify đơn lẻ là bằng chứng toàn bộ header chưa bị sửa.

3.2.5. Mô-đun thẩm định chữ ký và khôi phục tệp gốc

Luồng open() gồm kiểm tra phong bì và chữ ký; mở khóa bí mật bên nhận; giải header.key bằng RSA-OAEP lấy khóa AES; giải payload.encrypted_data bằng AES-CBC với payload.iv; kiểm tra protocol/control; giải Base64 tệp; đối chiếu kích thước và SHA-256; lưu tài liệu và biên nhận. REST endpoint /api/envelopes/receive lấy mật khẩu từ RECEIVE_KEY_PASSPHRASE trong cấu hình cục bộ của chính bên nhận. Phía A chỉ gửi phong bì, không cần biết hoặc truyền mật khẩu khóa bí mật của B. Lỗi mật mã được trả bằng thông báo chung, không lộ chi tiết OpenSSL.

StorageService.accept() ghi byte gốc vào MongoDB GridFS và tạo bản ghi receipts gồm request_id, các đơn vị, thời gian, tên tệp, trích yếu, SHA-256, kích thước và fileId. Chỉ mục duy nhất trên ownerId/request_id khiến hai yêu cầu đồng thời chỉ có một lần chấp nhận. Nếu yêu cầu thua đã tải tệp lên GridFS, tệp của yêu cầu đó được xóa để tránh dư dữ liệu. Lịch sử tồn tại sau khi ứng dụng khởi động lại; gói sai chữ ký, sai mật khẩu hoặc sai control không được ghi nhận. Biên nhận được lưu trước phản hồi HTTP nên chưa bảo đảm trình duyệt nhận đúng một lần khi kết nối đứt. Các test lõi độc lập MongoDB sử dụng dấu tệp cục bộ, còn demo và benchmark trong báo cáo đều chạy với MongoDB thật.

Ghi tệp GridFS và ghi receipts là hai thao tác, chưa phải một giao dịch bao trùm cả hai. Nếu mất phản hồi sau khi MongoDB đã commit, chương trình đọc lại request_id/fileId để xác nhận kết quả và giữ nguyên tệp đã nhận. Nếu không xác định được trạng thái ghi, tệp được giữ để đối soát thay vì xóa có nguy cơ mất dữ liệu. Trường hợp tiến trình dừng hoặc mất mạng kéo dài vẫn có thể để lại tệp chưa có biên nhận; cơ chế đối soát và dọn dữ liệu sau sự cố cần được phát triển thêm.

Dữ liệu	Vị trí lưu	Nội dung chính
Khóa bí mật	data/<đơn-vị>/identity/private.pem	PKCS#8 mã hóa; chỉ đơn vị sở hữu sử dụng
Khóa đối tác	MongoDB: peers	ownerId, unitId, fingerprint, publicKeyPem

Dữ liệu	Vị trí lưu	Nội dung chính
Biên nhận	MongoDB: receipts	UUID, các đơn vị, timestamp, tên tệp, trích yếu, SHA-256, fileId
Tài liệu gốc	MongoDB GridFS	documents.files và documents.chunks; liên kết bằng fileId

Tài liệu được phục hồi ở dạng rõ trong kho nhận, vì vậy MongoDB thuộc môi trường nội bộ đáng tin cậy. Bản trình diễn chỉ mở cơ sở dữ liệu và ứng dụng trên loopback, dùng tệp mẫu. Việc phân quyền đọc, mã hóa dữ liệu lưu trữ và sao lưu chưa thuộc phạm vi thử nghiệm. GridFS tránh phụ thuộc giới hạn một BSON document, nhưng ứng dụng vẫn giới hạn tệp đầu vào 10 MiB vì xử lý mật mã trong RAM.

3.3. Giao diện và kết quả thực thi chương trình

Phương thức / endpoint	Mục đích thao tác
POST /api/keys/generate	Sinh cặp khóa RSA cho đơn vị hiện tại
POST /api/keys/import; GET /api/keys/public	Nhập khóa có sẵn, xuất khóa công khai PEM
POST /api/peers	Đối chiếu vân tay và lưu khóa đối tác vào MongoDB
POST /api/envelopes/seal	Chọn tệp, nhập trích yếu, mã hóa và tạo chữ ký
POST /api/envelopes/verify	Kiểm tra chữ ký encrypted_data; chưa xác nhận control hoặc ghi nhận UUID
POST /api/envelopes/open	Thao tác mở gói của người nhận và tải tệp gốc
POST /api/envelopes/receive	API liên thông nhận JSON, mở và lưu MongoDB, trả biên nhận
GET /api/documents/:id; GET /api/storage/status	Tải tài liệu GridFS; kiểm tra số bản ghi nhận

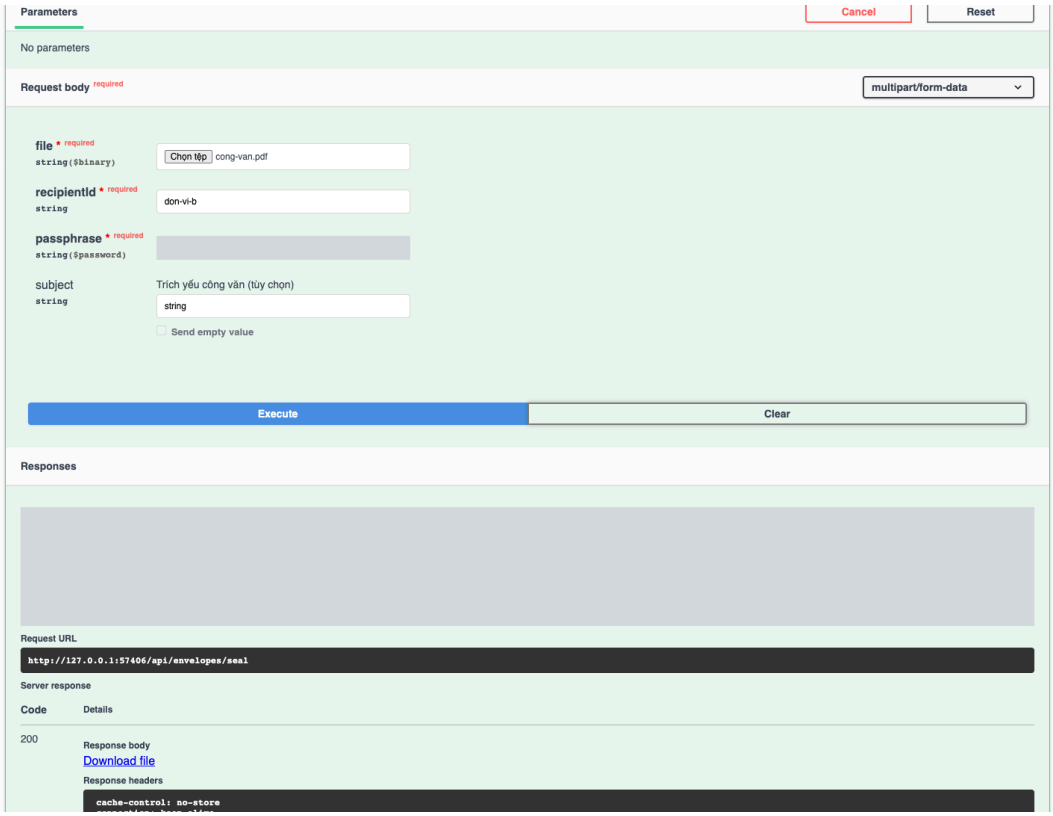
3.3.1. Giao diện thao tác của bên gửi

Giao diện thao tác sử dụng Swagger UI do NestJS sinh tự động, gồm biểu mẫu chọn tệp, nhập tham số và tải kết quả. Trước hết chạy docker compose up -d mongodb để khởi tạo MongoDB 8.0 trên cổng loopback 27018, sau đó npm ci và npm run build. Hai đơn vị chạy ở hai terminal với UNIT_ID, DATA_DIR và PORT riêng:

```
UNIT_ID=don-vi-a DATA_DIR=./data/don-vi-a PORT=3000 npm start
UNIT_ID=don-vi-b DATA_DIR=./data/don-vi-b PORT=3001 npm start
```

Để B tự động mở gói qua REST, đặt RECEIVE_KEY_PASSPHRASE trong cấu hình cục bộ của tiến trình B bằng đúng mật khẩu đã dùng khi sinh hoặc nhập khóa B. Nếu chưa cấu hình giá trị này, endpoint receive trả HTTP 503; người vận hành vẫn có thể dùng biểu mẫu open và nhập mật khẩu trực tiếp. Không đặt mật khẩu B trong gói JSON hoặc gửi mật khẩu này từ A.

Tại /docs của mỗi bên, người dùng sinh khóa bằng POST /api/keys/generate, trao đổi khóa công khai và ghim bằng POST /api/peers. Khóa bí mật được nạp từ kho cục bộ khi người dùng nhập mật khẩu. Tại bên A, biểu mẫu POST /api/envelopes/seal có trường file để chọn tài liệu, recipientId để chỉ định bên B và passphrase để mở khóa ký. Bấm Execute tạo phong bì JSON; Download file tải kết quả.



Hình 3.1. Gửi tệp PDF và tạo phong bì tại đơn vị A; các trường chứa mật khẩu đã được che.

3.3.2. Cấu trúc tệp package.json thực tế sau đóng gói

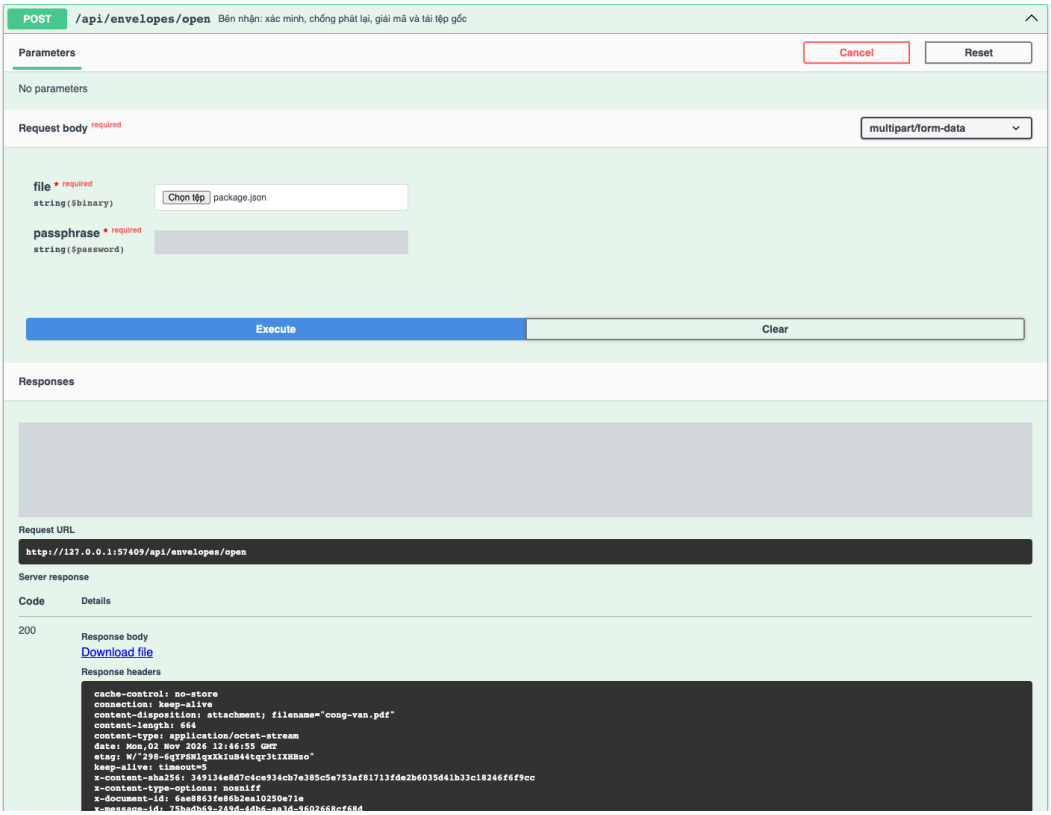
Tên package.json trong đề cương được hiểu là tệp gói tin trao đổi, khác với package.json cấu hình npm ở thư mục gốc. Chương trình mặc định đặt tên envelope-<UUID>.json để tránh nhầm lẫn. Bản rút gọn dưới đây lấy header từ gói PDF đã thực sự tạo; các chuỗi Base64 được rút ngắn để trình bày, không phải gói có thể dùng trực tiếp để giải mã.

```
{
  "header": {
    "request_id": "881c438f-3186-4ac5-abb7-6663c1ef3268",
    "sender_id": "don-vi-a",
    "receiver_id": "don-vi-b",
    "timestamp": "2026-11-02T14:14:46.967Z",
    "key": "WVAVG7IpWqSZbhYIcOyXgvh4... (rút gọn)"
  },
  "payload": {
    "encrypted_data": "lAh2S/BZzkWB1Nf0X74a91DF... (rút gọn)",
    "iv": "Um00M+HBKEYHiXu5MmwdvQ==",
    "signature": "kzg5FmLdWTnbKpW0UgmTA/PD... (rút gọn)"
  }
}
```

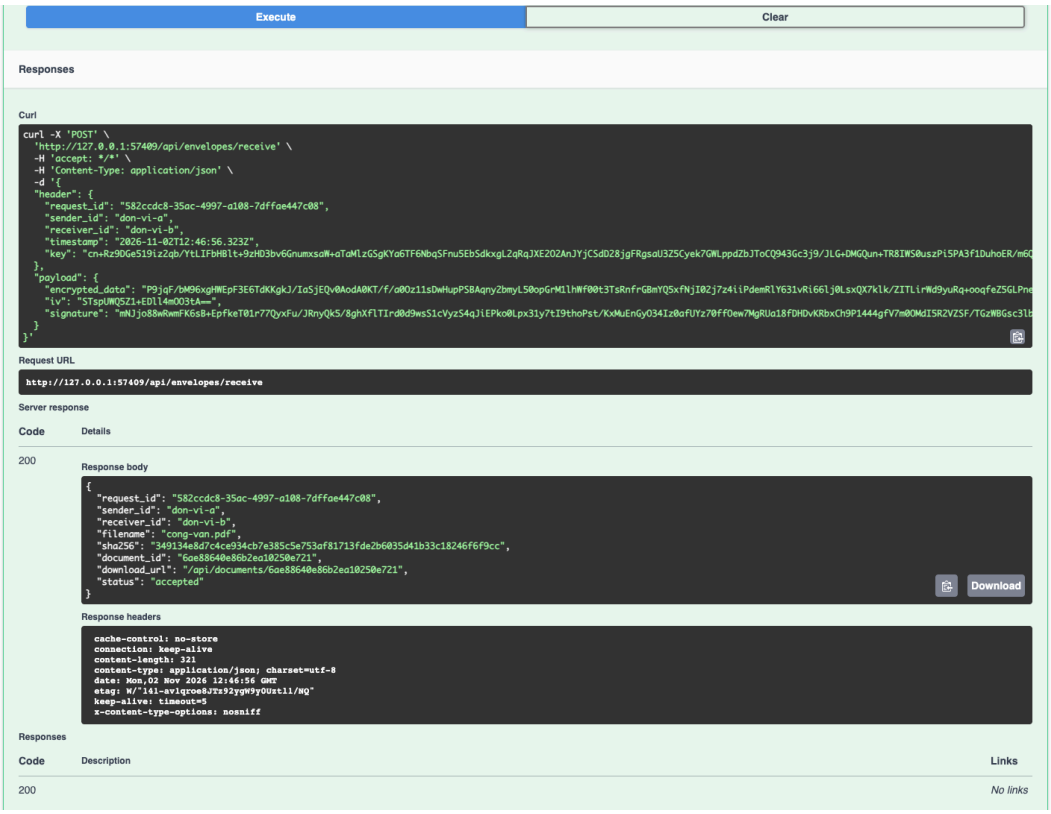

Gói đầy đủ và tệp được phục hồi nằm tại artifacts/demo/2026-11-02T14-14-46-801Z. JSON bản rõ bên trong chứa {protocol, control, subject, filename, mimeType, size, sha256, data}; các thông tin này không lộ ở header. Header công khai vẫn bộc lộ danh tính đơn vị, thời điểm và kích thước gần đúng của trao đổi.

3.3.3. Giao diện thao tác của bên nhận

Tại B, giao diện POST /api/envelopes/open cho phép chọn tệp phong bì, nhập mật khẩu của B và tải tài liệu gốc bằng Download file. Phản hồi có X-Content-SHA256, X-Message-Id, X-Verified-Sender và X-Document-Id để đối chiếu. Đối với liên thông máy chủ đúng mục 2.4, A gửi nội dung JSON tới POST /api/envelopes/receive của B. B xác minh, giải mã và lưu MongoDB, sau đó trả biên nhận gồm request_id, document_id, sha256 và download_url. GET download_url đọc lại byte từ GridFS. GET /api/storage/status cung cấp bằng chứng kết nối và số bản ghi. Gửi lại cùng request_id nhận HTTP 409.

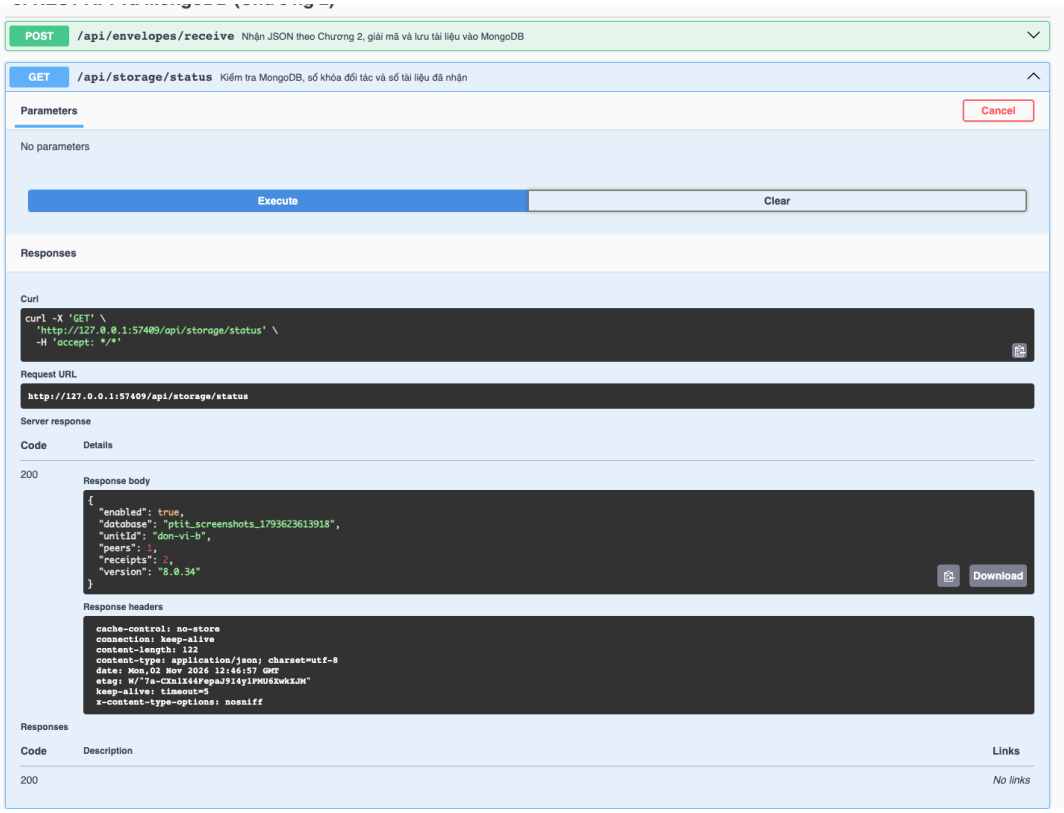


Hình 3.2. Đơn vị B mở phong bì và tải tài liệu gốc qua Swagger UI.



Hình 3.3. Tiếp nhận phong bì JSON qua REST API và trả biên nhận lưu trữ.

Trong lượt demo dùng làm dữ liệu báo cáo, MongoDB 8.0.34 tại cơ sở dữ liệu ptit_demo_1793628886801 ghi nhận 1 khóa đối tác của B và 3 tài liệu đã tiếp nhận. Collection peers tách danh mục bằng ownerId; receipts có unique index ownerId/request_id; documents.files và documents.chunks lưu dữ liệu GridFS. Khóa bí mật không được đưa vào các collection này.



Hình 3.4. Trạng thái và số bản ghi MongoDB của một phiên trình diễn giao diện.

Ảnh giao diện được chụp trong một phiên riêng dùng cơ sở dữ liệu tạm riêng, vì vậy tên DB hoặc số lần thao tác trên ảnh có thể khác lượt demo tổng hợp. Tệp artifacts/demo-results.json là nguồn chính cho bảng kết quả định lượng.

3.4. Kịch bản kiểm thử và đánh giá an toàn

Các kịch bản gồm kiểm thử mật mã ở lớp service và kiểm thử tích hợp HTTP/MongoDB. Khi sửa gói, chỉ thay đúng thành phần khảo sát, giữ nguyên các thành phần khác để xác định nguyên nhân bị từ chối. Kết quả được kiểm tra bằng mã phản hồi, đối chiếu byte và số bản ghi lưu trữ. Lỗi chữ ký không được coi là bằng chứng đã xác thực tất cả các trường điều khiển.

Bộ kiểm thử được chạy bằng npm test tại 2026-11-02T14:13:02.559Z. Kết quả: 42/42 ca đạt, 0 ca thất bại. Bằng chứng nằm trong artifacts/test-results.tap và

artifacts/test-results.json. Lệnh npm run demo kiểm tra thêm luồng HTTP thực tế với hai máy chủ tạm tại 2026-11-02T14:14:47.085Z.

3.4.1. Kịch bản 1: Luồng truyền nhận chuẩn

Đơn vị A mã hóa và ký công văn PDF bằng khóa của mình và khóa công khai của B. B kiểm tra bằng khóa công khai đã ghim của A, sau đó giải mã bằng khóa bí mật của B. Chương trình so sánh trực tiếp từng byte của tài liệu khôi phục với đầu vào và tính SHA-256. Cả ba tệp trong demo đều trùng khớp hoàn toàn; tệp nhị phân qua hai máy chủ HTTP cũng được kiểm tra trong bộ test.

SHA-256 của PDF khôi phục: 349134e8d7c4ce934cb7e385c5e753af81713fde2b6035d41b33c18246f6f9cc.

3.4.2. Kịch bản 2: Chống can thiệp dữ liệu

Kịch bản demo đảo một bit trong ciphertext rồi giữ nguyên chữ ký. Bên nhận trả HTTP 401 và không giải mã. Bộ test còn sửa IV, header.key, chữ ký, người nhận, UUID và thời gian. Bản mã/chữ ký bị sửa nhận HTTP 401; header hoặc IV không khớp control bị từ chối ở bước kiểm tra cấu trúc, giải mã hoặc đối chiếu control bằng HTTP 400. Một kiểm thử cố tình cung cấp mật khẩu sai cùng gói đã bị sửa nhưng vẫn nhận lỗi chữ ký trước, xác nhận thứ tự xác minh trước giải mã.

3.4.3. Kịch bản 3: Chống mạo danh bên gửi

Kẻ giả mạo dùng cặp RSA khác ký lại phong bì nhưng vẫn khai báo người gửi là A. B kiểm tra bằng khóa A đã ghim nên chữ ký không hợp lệ; demo nhận HTTP 401. Khóa chưa được ghim, vân tay sai hoặc ý định thay khóa đã ghim cũng bị từ chối. Cơ chế này phụ thuộc việc trao đổi vân tay ban đầu đúng: không thể xác thực danh tính nếu người dùng tự ghim khóa giả ngay từ đầu.

3.4.4. Kịch bản 4: Nhiều định dạng và các trường hợp biên

Kiểm thử HTTP bổ sung tên tệp tiếng Việt có dấu để kiểm tra cả nội dung lẫn tên tệp khi tải về. API receive thiếu body phải trả lỗi đầu vào HTTP 400. Một kiểm thử gây lỗi giả lập sau thao tác ghi MongoDB thật xác nhận rằng mất phản hồi không làm chương trình xóa tệp mà biên nhận đã tham chiếu tới.

Kịch bản bổ sung	Điểm kiểm tra	Kết quả mong đợi
Đổi request_id/timestamp của gói cũ	Đối chiếu control sau giải mã	HTTP 400, không tạo biên nhận mới

Kịch bản bổ sung	Điểm kiểm tra	Kết quả mong đợi
Sai mật khẩu khóa nhận	Mở khóa PEM	HTTP 400; vẫn nhận được khi thử lại đúng mật khẩu
Timestamp lệch ±301 giây	Cửa sổ thời gian ±300 giây	HTTP 400
Hai yêu cầu cùng UUID đồng thời	Unique index MongoDB	Một thành công, một HTTP 409; không dư tệp GridFS
Kết nối MongoDB mới	Kho bền vững	Đọc được khóa công khai và lịch sử nhận
Nhập khóa PKCS#1	Lưu PKCS#8 mã hóa	Giữ nguyên vân tay; không ghi đè khóa đã có

Tệp thử	Tệp gốc (byte)	Phong bì HTTP (byte)	Đối chiếu byte
cong-van.pdf	664	3230	Trùng khớp
cong-van.docx	9956	19830	Trùng khớp
anh-mau.png	69	2166	Trùng khớp

PDF và DOCX được tạo riêng cho thử nghiệm; PNG là ảnh 1×1 pixel có cấu trúc và checksum hợp lệ. Hệ thống bảo vệ byte của tệp, không bóc tách hay kiểm chứng nội dung nghiệp vụ bên trong PDF/Word/ảnh. Các mẫu nhỏ này chứng minh tính độc lập định dạng của luồng mã hóa, chưa đại diện cho mọi biến thể tệp thực tế.

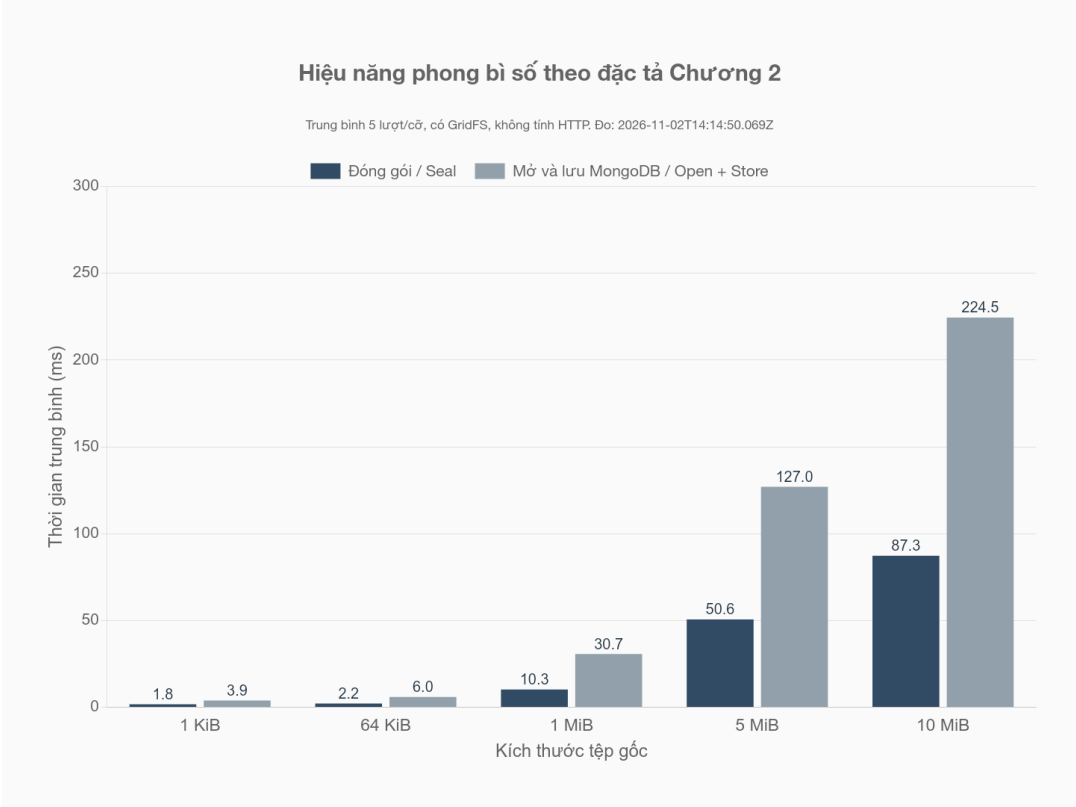
Bộ test bổ sung tệp rỗng, kích thước 1/15/16/17 byte và 64 KiB, JSON/Base64 lỗi, trường dư, schema không hỗ trợ, sai mật khẩu, gói ngoài cửa sổ timestamp, ngày gửi tương lai, sai người nhận và hai lần mở đồng thời. Gói phát lại qua HTTP bị từ chối với mã 409. Kiểm thử khởi tạo lại service vẫn phát hiện dấu nhận cũ, chứng minh lịch sử không chỉ nằm trong RAM.

3.4.5. Đo thời gian thực thi và độ tăng dung lượng

Phép đo được chạy tại 2026-11-02T14:14:50.069Z, với một lượt khởi động làm nóng và 5 lượt cho mỗi kích thước. Đầu vào là byte ngẫu nhiên; mỗi lượt tạo UUID, khóa phiên và IV mới. Bảng dùng thời gian trung bình ở lớp service, gồm đọc khóa PEM, JSON/Base64, mật mã, truy vấn khóa MongoDB và lưu tệp GridFS; không bao gồm truyền mạng, tạo dữ liệu đầu vào hoặc sinh cặp RSA ban đầu. Số liệu thô từng lượt và trung vị nằm trong artifacts/benchmark.json; bảng CSV nằm trong artifacts/benchmark.csv.

Tệp gốc	Phong bì gọn (byte)	Tăng dung lượng	Gửi TB (ms)	Nhận TB (ms)
1 KiB	3767	267.87%	1.78	3.93
64 KiB	118455	80.75%	2.19	5.96
1 MiB	1866079	77.96%	10.26	30.68
5 MiB	9322635	77.82%	50.63	126.95
10 MiB	18643295	77.80%	87.29	224.46

Với tệp 10 MiB (10.485.760 byte, lớn hơn 10 MB thập phân), tổng thời gian trung bình đóng gói và mở/lưu là 311.75 ms, tương đương 0.312 giây. Trên đúng cấu hình thử nghiệm này, kết quả đáp ứng chỉ tiêu 1,5 giây ở mục 2.1.3. Phép đo chưa tính HTTP giữa hai máy vật lý, do đó không kết luận chỉ tiêu luôn được đáp ứng trên mọi cấu hình hoặc môi trường mạng.



Hình 3.5. Thời gian trung bình đóng gói và mở/lưu MongoDB theo kích thước tệp.

Benchmark dùng JSON gọn, trong khi API HTTP xuất JSON thụt lề để dễ đọc, vì vậy kích thước hai dạng có thể khác. Không dùng kết quả tổng luồng này để suy ra thông lượng thuần AES hoặc khả năng phục vụ đồng thời. Năm lượt trên một máy chỉ là phép đo mô tả của bản thực nghiệm.

Với n byte đầu vào, một lớp Base64 tạo 4 × ceil(n/3) ký tự. Luồng mục 2.4.2 Base64 tệp trước khi tạo JSON bản rõ, rồi Base64 thêm bản mã để có encrypted_data. Khi n lớn, tỷ lệ tiến gần $(4/3)^2 = 16/9$, tức tăng khoảng 77,8%, cộng metadata, khóa bọc, chữ ký và padding. Mức 33,3% tại mục 1.6.3 vẫn đúng cho một phép Base64 riêng lẻ, nhưng không phải tổng chi phí phong bì hai lớp. Với tệp nhỏ, các trường control/header và chữ ký chiếm tỷ lệ lớn hơn.

Kết quả cho thấy bản thực nghiệm đáp ứng việc giữ bí mật nội dung, phát hiện can thiệp, kiểm tra quyền kiểm soát khóa người gửi và từ chối phát lại trong phạm vi thử nghiệm. Các giới hạn còn lại gồm xử lý toàn tệp trong RAM, giới hạn 10 MiB, trao đổi khóa thủ công, chưa có PKI, chứng thư, phân quyền người dùng, TLS hoặc thiết bị ký chuyên dụng. Một cặp RSA dùng cho hai mục đích để bám mô hình bài tập; phiên bản triển khai cần tách khóa theo mục đích và đánh giá giao thức đầy đủ. Chữ ký hiện tại không tự chứng minh việc bên nhận đã nhận tài liệu hoặc giá trị pháp lý của danh tính; các nội dung đó cần hạ tầng và biên nhận thích hợp.

Các ca đã thực hiện phát hiện cả thay đổi bản mã, giả mạo chữ ký và tái sử dụng bản mã kèm UUID mới. Đây là bằng chứng cho các đường xử lý được khảo sát, chưa phải chứng minh an toàn hình thức hoặc kiểm toán độc lập. Bản sao control là phần bổ sung của cách cài đặt theo đặc tả Chương 2; một giao thức triển khai thực tế cần được thẩm định đầy đủ, quản lý khóa theo từng mục đích và xác thực danh tính tin cậy.

Tài liệu tham khảo của Chương 3

[1] Node.js v22, Crypto API. https://nodejs.org/docs/latest-v22.x/api/crypto.html

[2] NestJS, OpenAPI (Swagger). https://docs.nestjs.com/openapi/introduction

[3] NestJS, File upload. https://docs.nestjs.com/techniques/file-upload

[4] NIST SP 800-38A, mục 6.2 và Phụ lục C. https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38a.pdf

[5] IETF RFC 8017, PKCS #1 v2.2, mục 7.1 và 8.2. https://www.rfc-editor.org/rfc/rfc8017

[6] MongoDB, Node.js Driver: GridFS và Indexes. https://www.mongodb.com/docs/drivers/node/current/crud/gridfs/ ; https://www.mongodb.com/docs/drivers/node/current/indexes/